ASSIGNMENT NO.8: E-Commerce Order Analytics System

Author: Snehal A. Bhosale
College: Sanjivani College of Engineering, Kopargaon
Email : snehalbhosale1807@gmail.com
CEI ID: CT_CSI_DE_1177

Dataset:

customers.csv
products.csv
orders.csv
order_items.csv

Techniques Used:

- Python
- Pandas
- Data Cleaning & Validation
- SQL
- MySQL
- Joins & Aggregations
- Window Functions
- CTEs
- Cohort Analysis
- Customer Segmentation
- CLI Reporting

Objective:

To build an end-to-end e-commerce analytics system using Python and SQL to clean, validate, analyze, and generate business insights from order data.

Step 1: Upload your CSVs to Google Colab

```
from google.colab import files

uploaded = files.upload()
```

```
Browse... 4 files selected.
customers.csv(text/csv) - 70922 bytes, last modified: n/a - 100% done
order_items.csv(text/csv) - 232794 bytes, last modified: n/a - 100% done
orders.csv(text/csv) - 107813 bytes, last modified: n/a - 100% done
products.csv(text/csv) - 24368 bytes, last modified: n/a - 100% done
Saving customers.csv to customers.csv
Saving order_items.csv to order_items.csv
Saving orders.csv to orders.csv
Saving products.csv to products.csv
```

```
import os

print(os.listdir())
```

```
['.config', 'orders.csv', 'order_items.csv', 'customers.csv', 'products.csv',
```

Step 2: Read the four CSVs

```
import pandas as pd

customers = pd.read_csv("customers.csv")
products = pd.read_csv("products.csv")
orders = pd.read_csv("orders.csv")
order_items = pd.read_csv("order_items.csv")

print("Customers:", customers.shape)
print("Products:", products.shape)
print("Orders:", orders.shape)
print("Order Items:", order_items.shape)
```

```
Customers: (1008, 5)
Products: (506, 5)
Orders: (2022, 5)
Order Items: (5094, 6)
```

Step 3: Inspect the datasets

```
print("CUSTOMERS")
display(customers.head())

print("PRODUCTS")
display(products.head())

print("ORDERS")
display(orders.head())

print("ORDER ITEMS")
```

```
display(order_items.head())
```

CUSTOMERS

	customer_id	customer_name	email	registration_dat
0	CUST0001	Richard Gupta	richard.gupta6019@example.com	2026-03-2
1	CUST0002	Melissa Brown	melissa.brown6524@example.com	2020-05-1
2	CUST0003	Isha Garcia	isha.garcia3476@example.com	2025-09-1
3	CUST0004	John Brown	john.brown5465@example.com	2020-05-3
4	CUST0005	Riya Wilson	riya.wilson1125@example.com	2025-03-2

PRODUCTS

	product_id	product_name	category	subcategory	cost_price
0	PROD0001	Sofa Set	Home	Furniture	14631.59
1	PROD0002	Women's Kurti	Clothing	Women	2754.31
2	PROD0003	Men's Jeans	Clothing	Men	1374.83
3	PROD0004	Wall Art	Home	Decor	1895.68
4	PROD0005	Men's T-Shirt	Clothing	Men	908.71

ORDERS

	order_id	customer_id	order_date	status	region_code
0	ORD000001	CUST0600	2023-07-02 01:55:29	DELIVERED	EAST
1	ORD000002	CUST0263	2025-05-31 15:13:40	SHIPPED	EAST
2	ORD000003	CUST0529	2023-07-19 15:34:28	DELIVERED	SOUTH
3	ORD000004	CUST0865	2023-09-18 14:48:02	DELIVERED	NORTH
4	ORD000005	CUST0255	2022-07-11 06:00:41	PLACED	EAST

ORDER ITEMS

	item_id	order_id	product_id	quantity	unit_price	discount_percent
0	ITEM000001	ORD000001	PROD0159	3	39556.37	11.3(
1	ITEM000002	ORD000001	PROD0275	2	895.20	8.0(
2	ITEM000003	ORD000001	PROD0280	3	11115.23	20.2(
3	ITEM000004	ORD000001	PROD0368	1	1736.46	22.3(
4	ITEM000005	ORD000002	PROD0272	1	1870.98	21.6(

```
print("Missing values")
```

```
print("\nCustomers:")
print(customers.isnull().sum())

print("\nProducts:")
print(products.isnull().sum())

print("\nOrders:")
print(orders.isnull().sum())

print("\nOrder Items:")
print(order_items.isnull().sum())
```

```
Missing values

Customers:
customer_id          0
customer_name        0
email                0
registration_date    0
customer_type        0
dtype: int64

Products:
product_id      0
product_name    0
category        0
subcategory     0
cost_price      0
dtype: int64

Orders:
order_id         0
customer_id    111
order_date       0
status           0
region_code      0
dtype: int64

Order Items:
item_id             0
order_id            0
product_id          0
quantity            0
unit_price          0
discount_percent    0
dtype: int64
```

Step 4: Clean Customers

```
import re
```

```
'

customers_clean = customers.copy()

# Remove duplicate customer IDs
customers_clean = customers_clean.drop_duplicates(subset=["customer_id"])

# Clean text columns
customers_clean["customer_name"] = (
    customers_clean["customer_name"]
    .astype(str)
    .str.strip()
)

customers_clean["email"] = (
    customers_clean["email"]
    .astype(str)
    .str.strip()
    .str.lower()
)

# Convert registration date
customers_clean["registration_date"] = pd.to_datetime(
    customers_clean["registration_date"],
    errors="coerce"
)

def validate_email(email):
    pattern = r"^[^@\s]+@[^@\s]+\.[^@\s]+$"
    return bool(re.match(pattern, email))

customers_clean["valid_email"] = customers_clean["email"].apply(validate_ema

invalid_emails = customers_clean.loc[
    ~customers_clean["valid_email"],
    "customer_id"
].tolist()

print("Invalid customer emails:", len(invalid_emails))
print(invalid_emails[:10])
```

```
Invalid customer emails: 30
['CUST0082', 'CUST0177', 'CUST0224', 'CUST0228', 'CUST0230', 'CUST0280', 'CUS
```

```
customers_clean = customers_clean.drop(columns=["valid_email"])
```

Step 5: Clean Products

```
products_clean = products.copy()
```

```
products_clean = products_clean.drop_duplicates(subset=["product_id"])

products_clean["product_name"] = (
    products_clean["product_name"]
    .astype(str)
    .str.strip()
    .str.title()
)

products_clean["category"] = (
    products_clean["category"]
    .astype(str)
    .str.strip()
    .str.title()
)

products_clean["subcategory"] = (
    products_clean["subcategory"]
    .astype(str)
    .str.strip()
    .str.title()
)

products_clean["cost_price"] = pd.to_numeric(
    products_clean["cost_price"],
    errors="coerce"
)

products_clean = products_clean[
    products_clean["cost_price"].notna()
]

print(products_clean.head())
```

```
  product_id   product_name  category subcategory  cost_price
0   PROD0001       Sofa Set      Home   Furniture    14631.59
1   PROD0002  Women'S Kurti  Clothing       Women     2754.31
2   PROD0003    Men'S Jeans  Clothing         Men     1374.83
3   PROD0004       Wall Art      Home       Decor     1895.68
4   PROD0005  Men'S T-Shirt  Clothing         Men      908.71
```

Step 6: Clean Orders

```
orders_clean = orders.copy()

orders_clean = orders_clean.drop_duplicates(subset=["order_id"])

orders_clean["customer_id"] = (
```

```
    orders_clean["customer_id"]
    .replace(["NULL", "null", "", " "], pd.NA)
)

orders_clean["order_date"] = pd.to_datetime(
    orders_clean["order_date"],
    errors="coerce",
    dayfirst=False
)

print("Missing customer IDs:",
      orders_clean["customer_id"].isna().sum())

print("Invalid order dates:",
      orders_clean["order_date"].isna().sum())
```

```
Missing customer IDs: 110
Invalid order dates: 103
```

Step 7: Clean Order Items

```
order_items_clean = order_items.copy()

order_items_clean = order_items_clean.drop_duplicates(
    subset=["item_id"]
)

order_items_clean["quantity"] = pd.to_numeric(
    order_items_clean["quantity"],
    errors="coerce"
)

order_items_clean["unit_price"] = pd.to_numeric(
    order_items_clean["unit_price"],
    errors="coerce"
)

order_items_clean["discount_percent"] = pd.to_numeric(
    order_items_clean["discount_percent"],
    errors="coerce"
)

# Keep negative quantities because they represent returns
# according to the assignment.

# Fix invalid discount values
order_items_clean.loc[
    (order_items_clean["discount_percent"] < 0) |
    (order_items_clean["discount_percent"] > 100),
```

```
    "discount_percent"
] = pd.NA

print(order_items_clean.head())
```

```
      item_id   order_id product_id  quantity  unit_price  discount_percent
0  ITEM000001  ORD000001   PROD0159         3    39556.37             11.30
1  ITEM000002  ORD000001   PROD0275         2      895.20              8.00
2  ITEM000003  ORD000001   PROD0280         3    11115.23             20.29
3  ITEM000004  ORD000001   PROD0368         1     1736.46             22.34
4  ITEM000005  ORD000002   PROD0272         1     1870.98             21.63
```

Step 8: Referential Integrity

```
valid_order_ids = set(orders_clean["order_id"].dropna())

invalid_order_items = order_items_clean[
    ~order_items_clean["order_id"].isin(valid_order_ids)
]

print("Invalid order references:",
      len(invalid_order_items))

display(invalid_order_items.head())
```

```
Invalid order references: 25
```

	item_id	order_id	product_id	quantity	unit_price	discount_perc
5049	ITEM005050	ORD986361	PROD0151	2	682.47	
5050	ITEM005051	ORD933750	PROD0292	2	226.90	1
5051	ITEM005052	ORD974403	PROD0063	1	70516.85	
5052	ITEM005053	ORD901895	PROD0037	2	2926.08	
5053	ITEM005054	ORD921680	PROD0369	1	1258.69	1

```
valid_product_ids = set(products_clean["product_id"])

invalid_product_items = order_items_clean[
    ~order_items_clean["product_id"].isin(valid_product_ids)
]

print("Invalid product references:",
      len(invalid_product_items))
```

```
Invalid product references: 0
```

Step 9: Prepare final cleaned data

```
order_items_clean = order_items_clean[
    order_items_clean["order_id"].isin(valid_order_ids)
]

order_items_clean = order_items_clean[
    order_items_clean["product_id"].isin(valid_product_ids)
]

orders_clean = orders_clean[
    orders_clean["customer_id"].isin(
        set(customers_clean["customer_id"])
    )
]
```

Step 10: Save cleaned CSVs

```
import os

os.makedirs("cleaned", exist_ok=True)

customers_clean.to_csv(
    "cleaned/customers_clean.csv",
    index=False
)

products_clean.to_csv(
    "cleaned/products_clean.csv",
    index=False
)

orders_clean.to_csv(
    "cleaned/orders_clean.csv",
    index=False
)

order_items_clean.to_csv(
    "cleaned/order_items_clean.csv",
    index=False
)

print("Cleaned files created successfully.")
```

```
Cleaned files created successfully.
```

```
from google.colab import files

files.download("cleaned/customers_clean.csv")
files.download("cleaned/products_clean.csv")
files.download("cleaned/orders_clean.csv")
files.download("cleaned/order_items_clean.csv")
```

Start coding or generate with AI.